INDEX

Experiment-01: Design and Implement Amplitude Shift Keying (ASK) modulation technique.

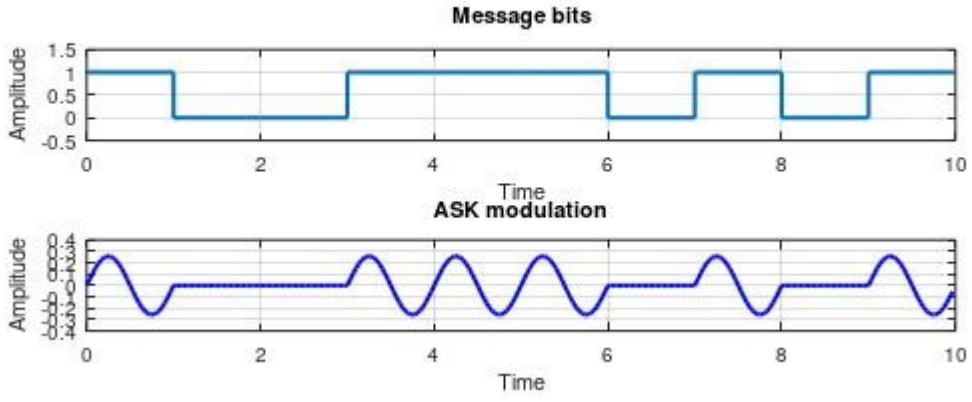
Source Code

```
clc;              % Clears the command window
close all;        % Closes all figure windows
clear all;        % Removes all variables


n=10;             % Number of bits to modulate
b=[1 0 0 1 1 1 0 1 0 1]; % Input bitstream (bits as message)
f1=1;             % Frequencies used
t=0:1/30:1-1/30;      % Time vector for one bit duration (30 samples per bit)


hf=sin(2*pi*f1*t);    % High-frequency sine wave (used for bit '1')
E= sum(hf.^2);        % Compute energy of the waveform
hf=hf/sqrt(E);        % Normalize the waveform to unit energy


lf=0*sin(2*pi*f1*t);  % Low-frequency signal (zero amplitude) for bit '0'
ask=[];           % Initialize empty ASK signal vector


% Loop through each bit in the message
for i=1:n
if b(i)==1
ask=[ask hf]; % Append high-frequency waveform for bit '1'
   else
ask=[ask lf]; % Append zero waveform for bit '0'
    end
end
```

```
figure(1)            % Create a new figure window
subplot(411)         % First subplot (top plot of 4)
stairs(0:10, [b(1:10) b(10)], 'linewidth',1.5) % Plot message bits as a step function
axis([0 10 -0.5 1.5])  % Set axis limits
title('Message bits');
grid on
xlabel('Time');
ylabel('Amplitude');
```

```
subplot(412)         % Second subplot (for ASK signal)
tb=0:1/30:10-1/30;     % Time vector for entire signal (10 bits)
plot(tb, ask(1:10*30), 'b', 'linewidth',1.5) % Plot ASK modulated signal
title('ASK modulation');
grid on
xlabel('Time');
ylabel('Amplitude');
```

Output:



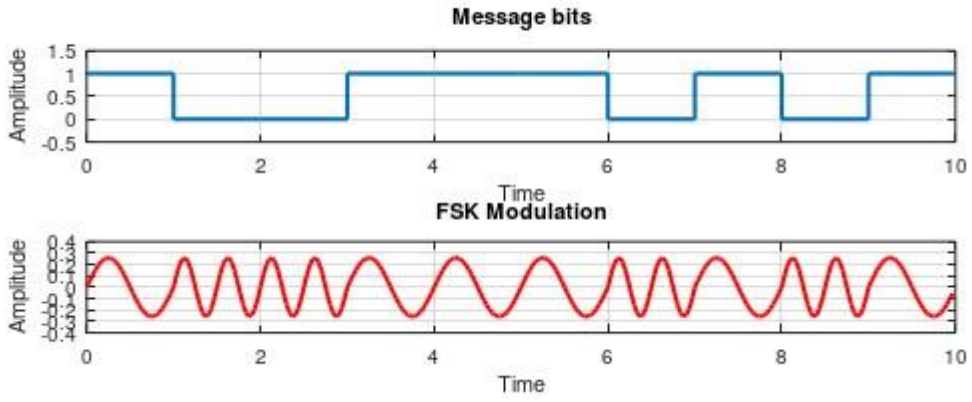
Experiment-02: Design and Implement Frequency Shift Keying (FSK) modulation technique.

Source Code:

```
clc;                % Clears the command window
close all;          % Closes all figure windows
clear all;          % Clears all variables
n = 10;             % Number of bits in the input message
b = [1 0 0 1 1 1 0 1 0 1];  % Input bitstream
f1 = 1;             % Frequency for bit '1'
f2 = 2;             % Frequency for bit '0'
t = 0:1/30:1-1/30;    % Time vector for one bit duration (30 samples per bit)


% Generate carrier wave for bit '1' with frequency f1
hf = sin(2*pi*f1*t);  % High frequency sine wave
E0 = sum(hf.^2);      % Energy of the high frequency signal
hf = hf / sqrt(E0);   % Normalize to unit energy


% Generate carrier wave for bit '0' with frequency f2
lf = sin(2*pi*f2*t);  % Low frequency sine wave
E1 = sum(lf.^2);      % Energy of the low frequency signal
lf = lf / sqrt(E1);   % Normalize to unit energy
fsk = [];           % Initialize empty array for FSK signal
% Loop to modulate each bit
  for i = 1:n
   if b(i) == 1
fsk = [fsk hf];  % Append high-frequency waveform for bit '1'
   else
      fsk = [fsk lf];  % Append low-frequency waveform for bit '0'
```

```
    end
      end
figure(1)                    % Create new figure window
subplot(411)                 % First subplot: Message bits
stairs(0:10, [b(1:10) b(10)], 'linewidth',1.5) % Stairs plot of message bits
axis([0 10 -0.5 1.5])        % Set axis limits
title('Message bits');
grid on
xlabel('Time');
ylabel('Amplitude');


subplot(412)                 % Second subplot: FSK signal
tb = 0:1/30:10-1/30;         % Time vector for the entire modulated signal
plot(tb, fsk(1:10*30), 'r', 'linewidth', 1.5) % Plot the modulated FSK waveform
title('FSK Modulation');
grid on;
xlabel('Time');
ylabel('Amplitude');
```

Output:



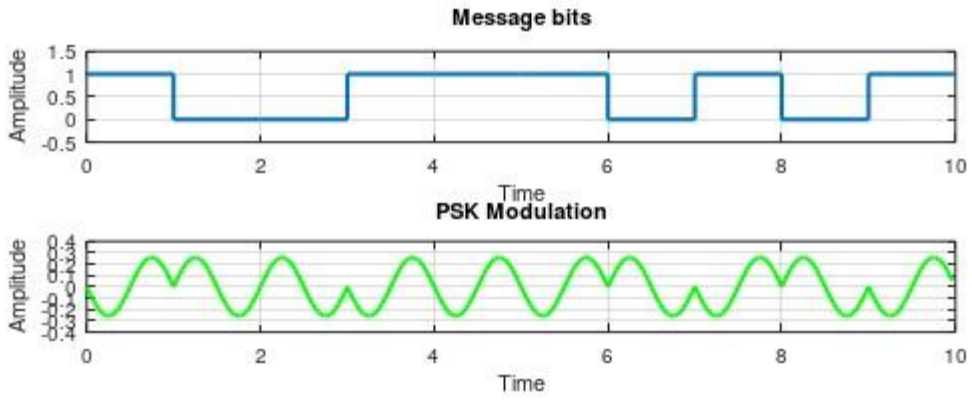
Experiment-03: Design and Implement Binary Phase Shift Keying (BPSK) modulation technique.

Source Code:

```
clc;                % Clear the command window
close all;          % Close all open figure windows
clear all;          % Clear all variables


n = 10;             % Number of bits in the bitstream
b = [1 0 0 1 1 1 0 1 0 1];  % Input binary message
f1 = 1;             % Carrier frequency
t = 0:1/30:1-1/30;    % Time vector for 1 bit duration with 30 samples
hf = sin(2*pi*f1*t);  % Generate sine wave carrier for 1 Hz
E = sum(hf.^2);       % Compute energy of one bit duration of the sine wave


% One with 180° phase shift for bit 1 and normal phase for bit 0
hf1 = -sin(2*pi*f1*t)/sqrt(E);  % Carrier with 180° phase shift (bit 1)
hf2 = sin(2*pi*f1*t)/sqrt(E);   % Normal carrier (bit 0)


psk = [];           % Initialize empty array to store PSK-modulated signal
% Loop through the bitstream and build the modulated signal
for i = 1:n
    if b(i) == 1
        psk = [psk hf1];  % For bit 1, append phase-shifted wave
     else
     psk = [psk hf2];  % For bit 0, append normal wave
   end
 end
```

```
figure(1)                       % Open figure window
subplot(411)                    % First subplot for message bits
stairs(0:10, [b(1:10) b(10)], 'linewidth', 1.5) % Stair-step plot of binary bits
axis([0 10 -0.5 1.5])           % Set axis limits
title('Message bits');          % Title for bitstream plot
grid on
xlabel('Time');
ylabel('Amplitude');


subplot(412)                    % Second subplot for PSK signal
tb = 0:1/30:10-1/30;            % Time vector for full 10-bit signal
plot(tb, psk(1:10*30), 'g', 'linewidth', 1.5) % Plot the modulated PSK signal in green
title('PSK Modulation');
grid on;
xlabel('Time');
ylabel('Amplitude');
```

Output:



Experiment-04: Design and Develop 4 to 1 Multiplexer.

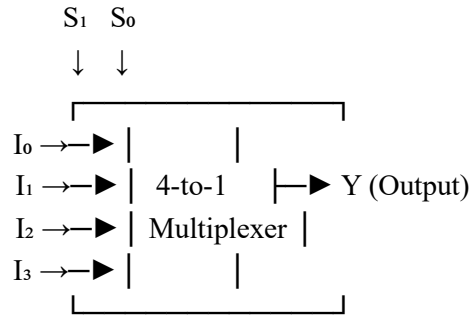
Source Code:

A Multiplexer (MUX) is a digital device that selects one input from several input lines and forwards it to a single output line. It acts like a digitally controlled switch. The selection of a particular input line is controlled by select lines. A multiplexer with 2^n input lines requires n selection lines.

A 4-to-1 Multiplexer has:

- 4 input lines: I_0, I_1, I_2, I_3
- 2 selection lines: S_0, S_1
- 1 output line: Y

The block diagram of a 4-to-1 MUX:



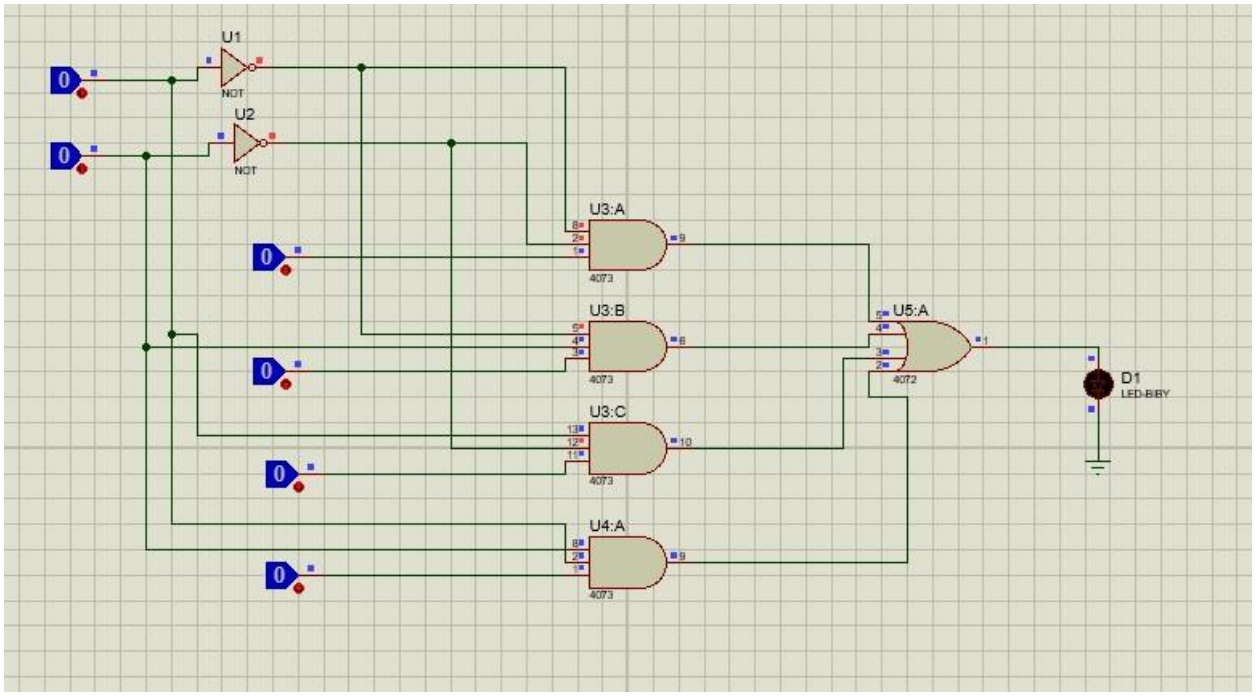
Truth Table of 4-to-1 Multiplexer

Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

The output of a 4-to-1 multiplexer can be expressed as:

$$Y = \overline{S}_1\overline{S}_0 \cdot I_0 + \overline{S}_1 S_0 \cdot I_1 + S_1\overline{S}_0 \cdot I_2 + S_1 S_0 \cdot I_3$$

Here's the logic circuit structure



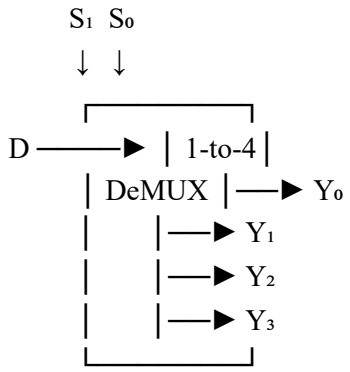
Experiment-05: Design and Develop 1 to 4 De-Multiplexer

Source Code: A De-Multiplexer (De-MUX) is a combinational circuit that takes one input and channels it to one of many outputs, depending on the select lines. It's the opposite of a multiplexer.

1-to-4 De-Multiplexer Overview

- Input Line: D (Data)
- Selection Lines: S_1 and S_0
- Output Lines: Y_0, Y_1, Y_2, Y_3

Only one output is active at a time, determined by the values of S_1 and S_0.



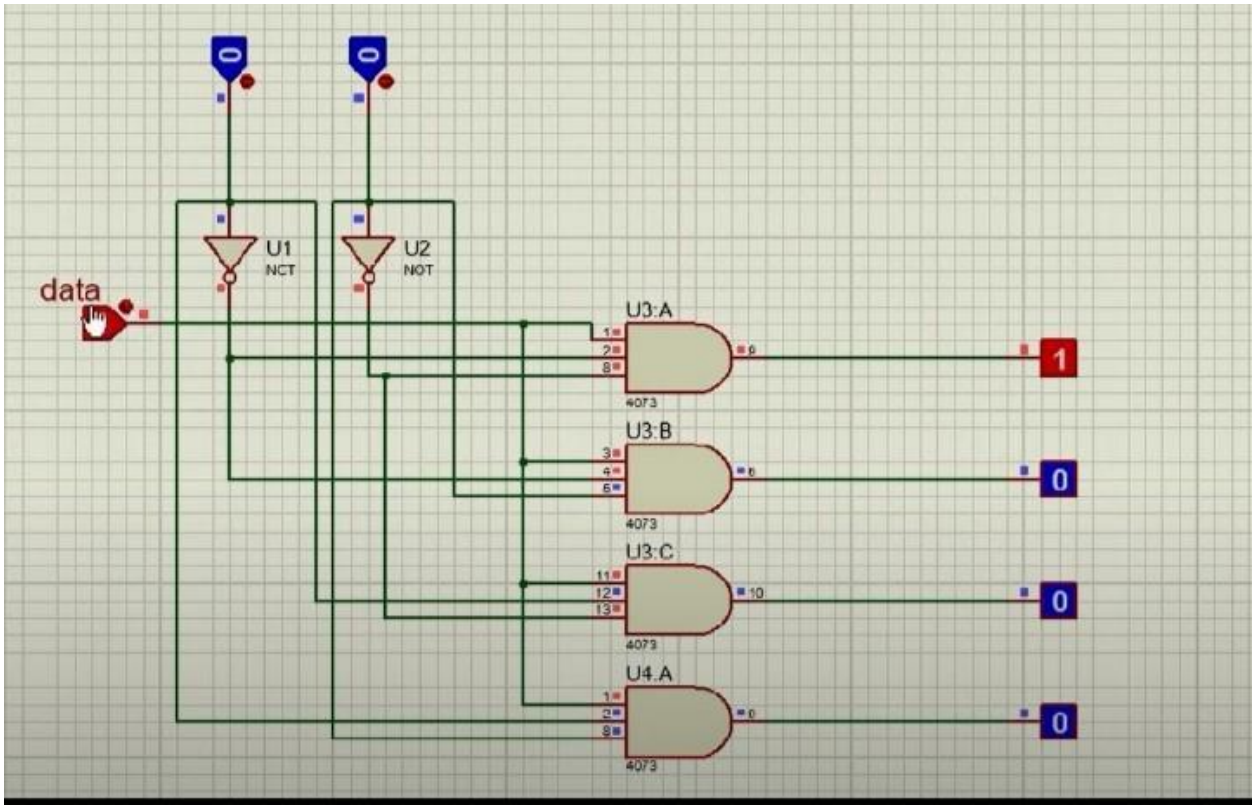
Truth Table:

Select Line		**Outputs**			
S_1	**S_0**	**Y_3**	**Y_2**	**Y_1**	**Y_0**
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Logic Expressions

- $Y_0 = D \cdot \overline{S}_1 \cdot \overline{S}_0$
- $Y_1 = D \cdot \overline{S}_1 \cdot S_0$
- $Y_2 = D \cdot S_1 \cdot \overline{S}_0$
- $Y_3 = D \cdot S_1 \cdot S_0$

Circuit Diagram:



Experiment-06: Design and implement Star and Tree topology.

Source Code:

Network topology refers to the arrangement of different elements in a computer network. Understanding different topologies helps in designing efficient and scalable networks. The topologies explored in this lab are -

1. Star Topology
2. Ring Topology
3. Bus Topology
4. Mesh Topology
5. Hybrid Topology

OBJECTIVES

●To design and implement star, tree topologies in Cisco Packet Tracer.

●To understand the characteristics and performance of each topology.

●To analyse the advantages and disadvantages of each topology

PROCEDURE

1. Star Topology:

In star topology, all devices are connected to a central switch or hub. This topology is easy to manage and troubleshoot.

Steps:

1. Open Cisco Packet Tracer and create a new project.
2. Add a switch to the workspace.
3. Connect PCs to the switch using network cables.
4. Assign IP addresses to each PC.
5. Verification: Ping from PC1 to PC2 and PC3 to verify connectivity.

2.Tree Topology:

Tree topology is created by forming a hierarchical structure, usually with a root switch (core) connected to intermediate switches (distribution/access), which connect to end devices. It resembles an inverted tree, combining multiple star topologies for scalability and organized network traffic.

Steps:

1. Open Cisco Packet Tracer and create a new workspace.
2. Add Devices-Place a main Switch at the top as the **Root Switch**.

   Place 2–3, or more, switches below the root to act as branch switches.

   Place end devices (PCs, Laptops) connected to each branch switch.
3. Connect devices using cables connect PCs to their respective branch

   switches using Straight-Through cables.
4. Configure IP Addresses to each pc or device.
5. Verify Connectivity-Open the command prompt on a PC and use the ping command to test connectivity with other devices (e.g., ping 192.168.1.5)

IMPLEMENTATION:

Main Router Configuration:

```
Switch(config)# interface GigabitEthernet0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 10
Switch(config-if)# no shutdown
.........
.........
.........
Switch(config)# interface GigabitEthernet0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 80
Switch(config-if)# no shutdown
```

Switch Configuration:

```
Router(config)# interface GigabitEthernet0/0
Router(config-if)# ip address 192.168.0.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/1
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/2
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/3
Router(config-if)# ip address 192.168.3.1 255.255.255.0
Router(config-if)# no shutdown
```

OUTPUT:

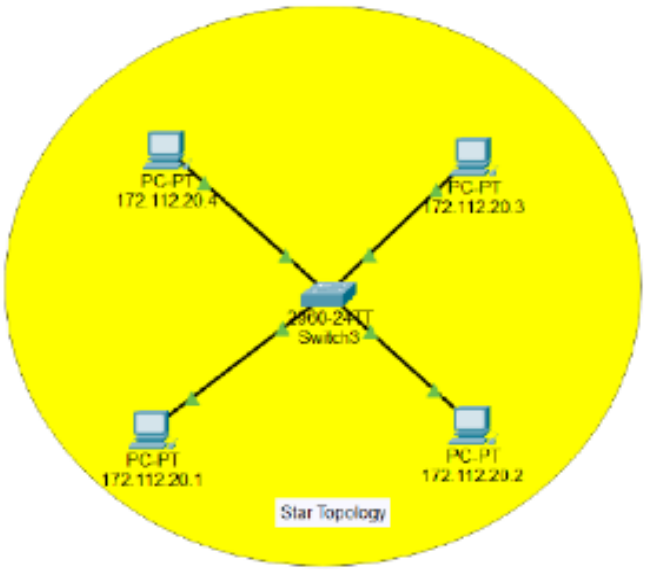
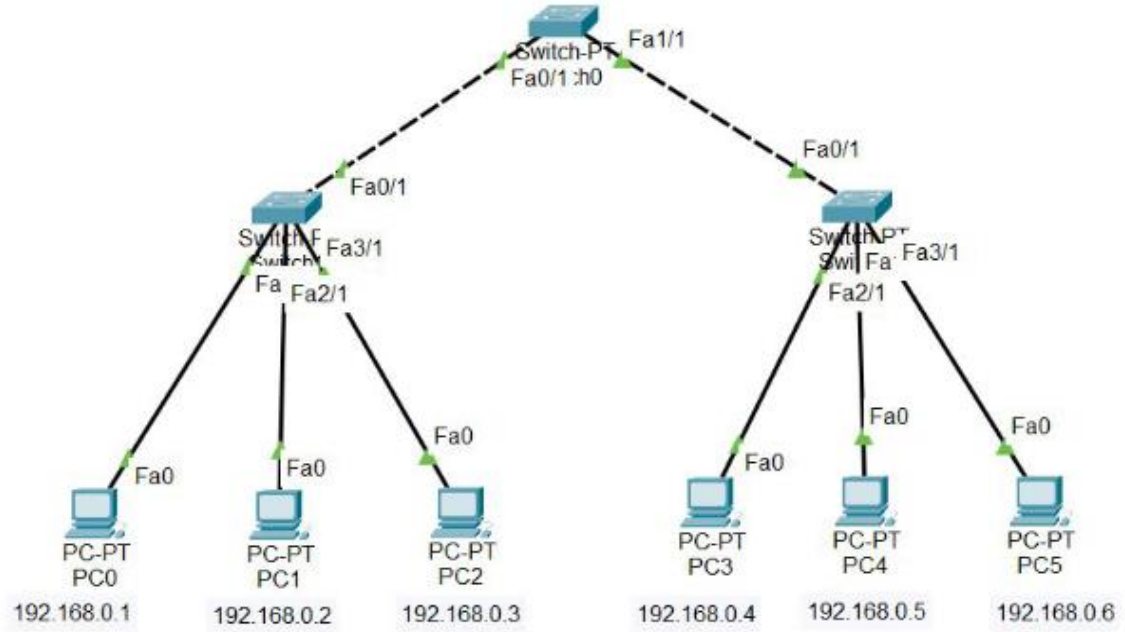


Fig: Star Topology



Experiment-06: Design and implement Mesh and Ring topology.

Source Code:

Network topology refers to the arrangement of different elements in a computer network. Understanding different topologies helps in designing efficient and scalable networks. The topologies explored in this lab are -

1. Star Topology
2. Ring Topology
3. Bus Topology
4. Mesh Topology
5. Hybrid Topology

OBJECTIVES

- To design and implement star, tree topologies in Cisco Packet Tracer.
- To understand the characteristics and performance of each topology.
- To analyse the advantages and disadvantages of each topology

PROCEDURE

1. Mesh Topology

In mesh topology, each device is connected to every other device. This provides high redundancy and reliability.

Steps:

1. Connect each PC to every other PC using network cables.
2. Assign IP addresses to each PC.
3. Verification: Ping from PC1 to PC2 and PC3 to verify connectivity.

2.Ring Topology

In ring topology, each device is connected to exactly two other devices, forming a circular data path.

Steps:

1. Add PCs and connect them in a ring using network cables.

2. Use a switch or hub to complete the ring if necessary.
3. Assign IP addresses to each PC.
4. Verification: Ping from PC1 to PC2 and PC3 to verify connectivity

IMPLEMENTATION:

Main Router Configuration:

```
Switch(config)# interface GigabitEthernet0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 10
Switch(config-if)# no shutdown
..........
..........
..........
Switch(config)# interface GigabitEthernet0/1
Switch(config-if)# switchport mode access
Switch(config-if)# switchport access vlan 80
Switch(config-if)# no shutdown
```

Switch Configuration:

```
Router(config)# interface GigabitEthernet0/0
Router(config-if)# ip address 192.168.0.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/1
Router(config-if)# ip address 192.168.1.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/2
Router(config-if)# ip address 192.168.2.1 255.255.255.0
Router(config-if)# no shutdown

Router(config)# interface GigabitEthernet0/3
Router(config-if)# ip address 192.168.3.1 255.255.255.0
Router(config-if)# no shutdown
```

OUTPUT:

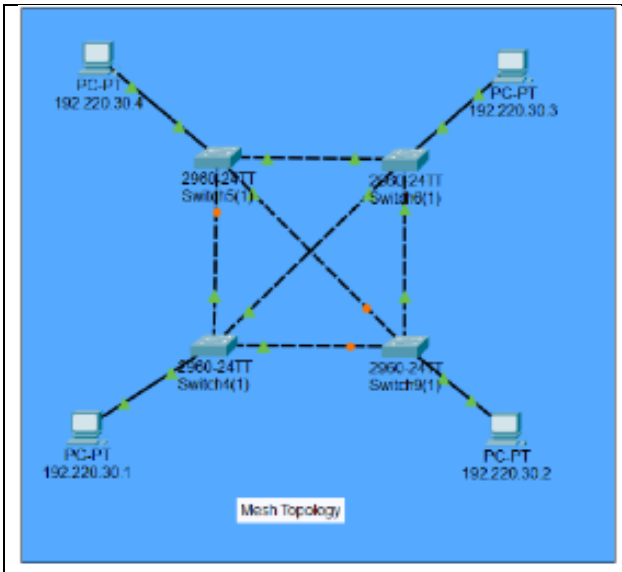


Fig: Mesh Topology

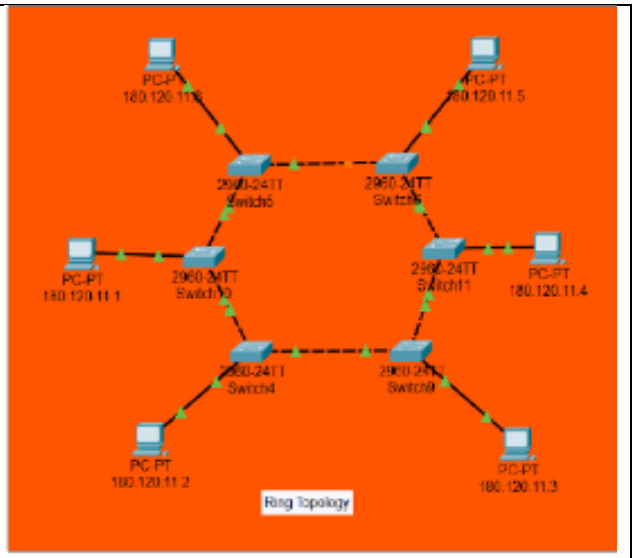


Fig: Ring Topology